Chiang Mai University

AnyaSpyAC

Suthana Kwaonueng, Theerada Siri, Sorawach Utas

2026-07-13

Contest (1)

template.cpp 80 lines

```
#include<bits/stdc++.h>
using namespace std;
void __print(int x) {cerr << x;}
void __print(long x) {cerr << x;}
void __print(long long x) {cerr << x;}
void __print(unsigned x) {cerr << x;}
void __print(unsigned long x) {cerr << x;}
void __print(unsigned long long x) {cerr << x;}
void __print(float x) {cerr << x;}
void __print(double x) {cerr << x;}
void __print(long double x) {cerr << x;}
void __print(char x) {cerr << '\'' << x << '\'';}
void __print(const char *x) {cerr << '\"' << x << '\"';}
void __print(const string &x) {cerr << '\"' << x << '\"';}
void __print(bool x) {cerr << (x ? "true" : "false");}

template<typename T, typename V>
void __print(const pair<T, V> &x);
template<typename T>
void __print(const T &x) {int f = 0; cerr << '{'; for (auto &i:
     x) cerr << (f++ ? ", " : ""), __print(i); cerr << "}";}
template<typename T, typename V>
void __print(const pair<T, V> &x) {cerr << '{'; __print(x.first
    ); cerr << ", "; __print(x.second); cerr << '}';}
void _print() {cerr << "]\n";}
template <typename T, typename... V>
void _print(T t, V... v) {__print(t); if (sizeof...(v)) cerr <<
     ", "; _print(v...);}
//#ifdef DEBUG
#define dbg(x...) cerr << "\e[91m"<<__func__<<":"<<__LINE__<<"
    [" << #x << "] = ["; _print(x); cerr << "\e[39m" << endl;
//#else
//#define dbg(x...)
//#endif

typedef long long ll;
typedef long double ld;
typedef complex<ld> cd;

typedef pair<int, int> pi;
typedef pair<ll,ll> pl;
typedef pair<ld,ld> pd;

typedef vector<int> vi;
typedef vector<ld> vd;
typedef vector<ll> vl;
typedef vector<pi> vpi;
typedef vector<pl> vpl;
typedef vector<cd> vcd;

//template<class T> using pq = priority_queue<T>;
//template<class T> using pqg = priority_queue<T, vector<T>,
    greater<T>>;
```

```
#define rep(i, a) for(int i=0;i<a;++i)
#define FOR(i, a, b) for (int i=a; i<(b); i++)
#define F0R(i, a) for (int i=0; i<(a); i++)
#define FORd(i,a,b) for (int i = (b)-1; i >= a; i--)
#define F0Rd(i,a) for (int i = (a)-1; i >= 0; i--)
#define trav(a,x) for (auto& a : x)
#define uid(a, b) uniform_int_distribution<long long>(a, b)(rng
    )

#define sz(x) (int)(x).size()
#define mp make_pair
#define pb push_back
//#define f first
//#define s second
#define lb lower_bound
#define ub upper_bound
#define all(x) x.begin(), x.end()
#define ins insert

template<class T> bool ckmin(T& a, const T& b) { return b < a ?
     a = b, 1 : 0; }
template<class T> bool ckmax(T& a, const T& b) { return a < b ?
     a = b, 1 : 0; }

mt19937 rng(chrono::steady_clock::now().time_since_epoch().
    count());

const char nl = '\n';
const int N = 2e5+3;
const int INF = INT_MAX;
const long long LINF = LLONG_MAX;

int main(){
    ios::sync_with_stdio(false);cin.tie(nullptr);
}
```

.vimrc 21 lines

```
"General editor settings
set tabstop=4
set nocompatible
set shiftwidth=4
set expandtab
set autoindent
set smartindent
set ruler
set showcmd
set incsearch
set shellslash
set number
set relativenumber
set cino+=L0

"keybindings for { completion, "jk" for escape, ctrl-a to
    select all
inoremap {<CR>  {<CR>}<Esc>O
inoremap {}     {}
imap jk         <Esc>
map <C-a> <esc>ggVG<CR>
set belloff=all
```

.bashrc 1 lines

```
export PATH=$PATH:~/scripts/
```

build.sh 1 lines

```
g++ -static -DLOCAL -lm -s -x c++ -Wall -Wextra -O2 -std=c++17
    -o $1 $1.cpp
```

stress.sh 22 lines

```
#!/usr/bin/env bash

for ((testNum=0;testNum<$4;testNum++))
do
    ./$3 > input
    ./$2 < input > outSlow
    ./$1 < input > outWrong
    H1=`md5sum outWrong`
    H2=`md5sum outSlow`
    if !(cmp -s "outWrong" "outSlow")
    then
        echo "Error found!"
        echo "Input:"
        cat input
        echo "Wrong Output:"
        cat outWrong
        echo "Slow Output:"
        cat outSlow
        exit
    fi
done
echo Passed $4 tests
```

Tanya (2)

2.1 Template & Utilities

pragma.h

Description: random pragma magic that I don't fucking know how this work, but it ocassionaly solve my stupid bad implementation code in the past(segtree constant factor), this should not be mandatory and only last resort

1021ce, 7 lines

```
//some one on cf told us that these are mostly enough
#pragma GCC target ("avx2")
#pragma GCC optimize ("O3")
#pragma GCC optimize ("unroll-loops")
//just in case have this one
#pragma GCC optimize("O3,unroll-loops")
#pragma GCC target("avx2,bmi,bmi2,popcnt,lzcnt")
```

2.2 Number Theory & Combinatoric

factorial.h

Description: use 2*N when do stars and bars

061239, 20 lines

```
long long fac[N], faci[N];

long long bigmod(long long a, long long n){
    long long res = 1;
    while(n>0){
        if(n&1){
            res = res * a % MOD;
        }
        a = a * a % MOD;
        n>>=1;
    }
    return res;
}

void cal_fac(){
    fac[0] = 1;
    for(int i=1;i<N;++i) fac[i] = fac[i-1] * i % MOD;
    faci[N-1] = bigmod(fac[N-1], MOD-2);
    for(int i=N-2;i>=0;--i) faci[i] = faci[i+1] * (i+1) % MOD;
}
```

Cnr.h

Description: this is ok

cc78ad, 11 lines

```
long long Cnr(int n, int k) {
    if(k<0 || n<k)return 0;
    if(k==0 or n==k)return 1;
    /*
    double res = 1;
    for (int i = 1; i <= k; ++i)
        res = res * (n - k + i) / i;
    return (long long)(res + 0.01);
    */
    return fac[n] * faci[k] % MOD * faci[n-k] % MOD;
}
```

lucas.h

Description: for large n and small modulo, the complexity is O(p * log-p(n)), p must be prime

10284e, 36 lines

```
// Standard nCr modulo p for small n and r
long long nCr_small(long long n, long long r, int p) {
    if (r < 0 || r > n) return 0;
    if (r == 0 || r == n) return 1;
    if (r > n / 2) r = n - r;

    long long num = 1, den = 1;
    for (int i = 0; i < r; ++i) {
        num = (num * (n - i)) % p;
        den = (den * (i + 1)) % p;
    }

    // Modular inverse using Fermat's Little Theorem (den^(p-2) % p)
    auto power = [&](long long base, long long exp) {
        long long res = 1;
        base %= p;
        while (exp > 0) {
            if (exp % 2 == 1) res = (res * base) % p;
            base = (base * base) % p;
            exp /= 2;
        }
        return res;
    };

    return (num * power(den, p - 2)) % p;
}

// Lucas Theorem Implementation
long long lucas(long long n, long long k, int p) {
    // Base cases
    if (k == 0) return 1;
    if (n < k) return 0;

    // Recursive step: nCr % p = ( (n/p)C(k/p) * (n%p)C(k%p) ) % p
    return (lucas(n / p, k / p, p) * nCr_small(n % p, k % p, p)) % p;
}
```

factoradix.h

Description: for that factorial base number finding nth permutation blablabla, also this using 0 base indexing, i.e. your 1st lexicographically permutation now is start at 0th

<ext/pb_ds/assoc_container.hpp>, <ext/pb_ds/tree_policy.hpp>

e835c3, 57 lines

```
typedef __gnu_pbds::tree<int, __gnu_pbds::null_type, less<int>,
     __gnu_pbds::rb_tree_tag, __gnu_pbds::
    tree_order_statistics_node_update> ordered_set;

ordered_set st;

vector<int> factoradix_to_permutation(int n, vector<int> &
    factoradix){
    st.clear(); for(int i=1;i<=n;++i) st.ins(i);
    int cl = 0;
    while((int)factoradix.size() < n){
        factoradix.pb(0); ++cl;
    }

    vector<int> res;
    for(int i=(int)factoradix.size()-1;i>=0;--i){
        auto it = st.find_by_order(factoradix[i]);
        res.pb(*it);
        st.erase(it);
    }

    while(cl--) factoradix.pop_back();

    return res;
}

vector<int> permutation_to_factoradix(int n, vector<int> &
    permutation){
    st.clear(); for(int i=1;i<=n;++i) st.ins(i);
    vector<int> res;
    for(int i=0;i<(int)permutation.size();++i){
        int cnt = st.order_of_key(permutation[i]);
        res.pb(cnt);
        auto it = st.find_by_order(cnt); st.erase(it);
    }
    reverse(all(res));
    while(!res.empty() and res.back() == 0)res.pop_back();

    return res;
}

vector<int> decimal_to_factoradix(ll n){
    vector<int> res;
    int d = 1;
    while(n > 0){
        res.pb(n % d);
        n/=d; ++d;
    }
    return res;
}

ll factoradix_to_decimal(vector<int> &cur){
    ll res = 0;
    ll d = 1;
    for(int i=0;i<(int)cur.size();++i){
        res += d*cur[i];
        d*=(i+1);
    }
    return res;
}
```

div-ceil-floor.h

Description: to stop c++ fucked up negative case of ceil, floor

2834e8, 9 lines

```
long long div_floor(__int128 a, long long b){
    __int128 q = a / b;
    __int128 r = a % b;
    if (r != 0 && ((a < 0) != (b < 0))) --q;
    return (long long)q;
}
long long div_ceil(__int128 a, long long b){
    return -div_floor(-a, b);
}
```

sieve.h

Description: construct is O(sqrt(n)), can use to find all prime factor of a number in O(log n)

13e484, 11 lines

```
const int M = 2e5+1;
vector<bool> is_prime(M+1, true);
void sieve(){
    is_prime[0] = is_prime[1] = false;
    for (int i = 2; i <= M; i++) {
        if (is_prime[i]) {
            for (ll j = (ll)i * i; j <= M; j += i)
                is_prime[j] = false;
        }
    }
}
```

linear-sieve.h

Description: lp[i] = minimum prime that divide i

8c3e32, 17 lines

```
vector<int> lp(N+1);
vector<int> pr;

void sieve(){
    for (int i=2; i <= N; ++i) {
        if (lp[i] == 0) {
            lp[i] = i;
            pr.push_back(i);
        }
        for (int j = 0; i * pr[j] <= N; ++j) {
            lp[i * pr[j]] = pr[j];
            if (pr[j] == lp[i]) {
                break;
            }
        }
    }
}
```

phi-sieve.h

Description: TODO

b4e4ad, 15 lines

```
//set value for M
const int M = 1e6;
vector<int> phi(M+1);

void phi_1_to_n() {
    for (int i = 0; i <= M; i++)
        phi[i] = i;

    for (int i = 2; i <= M; i++) {
        if (phi[i] == i) {
            for (int j = i; j <= M; j += i)
                phi[j] -= phi[j] / i;
        }
    }
}
```

phi-function.h

Description: TODO

fb8d57, 14 lines

```
//O(sqrt n)
int phi(int n) {
    int result = n;
    for (int i = 2; i * i <= n; i++) {
        if (n % i == 0) {
            while (n % i == 0)
                n /= i;
            result -= result / i;
        }
    }
    if (n > 1)
```

```
        result -= result / n;
    return result;
}
```

mobius-function.h

Description: TODO

c6a8f4, 10 lines

```
int mob[N+1];
void mobius() {
  mob[1] = 1;
  for (int i = 2; i <= N; i++){
    mob[i]--;
    for (int j = i + i; j <= N; j += i) {
      mob[j] -= mob[i];
    }
  }
}
```

miller-rabin.h

Description: TODO

980e40, 52 lines

```
//randomize primality test O(logn^2)
using u64 = uint64_t;
using u128 = __uint128_t;

u64 bigmod(u64 base, u64 e, u64 mod){
    u64 result = 1;
    base %= mod;
    while(e){
        if(e & 1)
            result = (u128)result * base % mod;
        base = (u128)base * base % mod;
        e >>= 1;
    }
    return result;
}

bool check_composite(u64 n, u64 a, u64 d, int s){
    u64 x = bigmod(a, d, n);
    if(x == 1 || x == n - 1)
        return false;
    for(int r = 1; r < s; r++){
        x = (u128)x * x % n;
        if(x == n - 1)
            return false;
    }
    return true;
}

//random number generator temporalily disable because my
    template already has it
//mt19937 rng(chrono::steady_clock::now().time_since_epoch().
    count());
long long rnd(long long x, long long y) {
  return uniform_int_distribution<long long>(x, y)(rng);
}

bool MillerRabin(u64 n, int iter=5){ // return true if n is
    probably prime, else return false.
    if(n < 4){
        return n == 2 || n == 3;
    }
    int s = 0;
    u64 d = n-1;
    while((d & 1) == 0){
        d >>= 1;
        s++;
    }
    for(int i = 0; i < iter; i++){
        long long a = rnd(2, n-2);
        if(check_composite(n, a, d, s))
            return false;
    }
    return true;
}
```

pollard-rho.h

Description: TODO

bc7c06, 20 lines

```
//randomize factorization in O(n^1/4 * logn)
long long mult(long long a, long long b, long long mod) {
    return (__int128)a * b % mod;
}

long long f(long long x, long long c, long long mod) {
    return (mult(x, x, mod) + c) % mod;
}

long long rho(long long n, long long x0=2, long long c=1) {
    long long x = x0, y = x0, g = 1;
    while (g == 1) {
        x = f(x, c, n);
        y = f(f(y, c, n), c, n);
        g = __gcd(abs(x - y), n);
    }

    if(g == n) return rho(n, uid(2, n-1), uid(-2, 2)); //uid is
        uniform long
    else return g;
}
```

discrete-log.h

Description: TODO

f7650d, 33 lines

```
// Returns minimum x for which a ^ x %m = b %m.
int solve(int a, int b, int m) {
    a %= m, b %= m;
    int k = 1, add = 0, g;
    while ((g = __gcd(a, m)) > 1) {
        if (b == k)
            return add;
        if (b % g)
            return -1;
        b /= g, m /= g, ++add;
        k = (k * 1ll * a / g) % m;
    }

    int n = sqrt(m) + 1;
    int an = 1;
    for (int i = 0; i < n; ++i)
        an = (an * 1ll * a) % m;

    map<int, int> vals;
    for (int q = 0, cur = b; q <= n; ++q) {
        vals[cur] = q;
        cur = (cur * 1ll * a) % m;
    }

    for (int p = 1, cur = k; p <= n; ++p) {
        cur = (cur * 1ll * an) % m;
        if (vals.count(cur)) {
            int ans = n * p - vals[cur] + add;
            return ans;
        }
    }
    return -1;
}
```

primitive-root.h

Description: TODO

c6d472, 34 lines

```
/*
    The following code assumes that the modulo p is a prime
        number.
    To make it works for any value of p, we must add
        calculation of phi(p)
*/
int powmod (int a, int b, int p) {
    int res = 1;
    while (b)
        if (b & 1)
            res = int (res * 1ll * a % p),  --b;
        else
            a = int (a * 1ll * a % p),  b >>= 1;
    return res;
}

int generator (int p) {
    vector<int> fact;
    int phi = p-1,  n = phi; // cal phi p by assuming it's
        prime
    for (int i=2; i*i<=n; ++i)
        if (n % i == 0) {
            fact.push_back (i);
            while (n % i == 0)
                n /= i;
        }
    if (n > 1)
        fact.push_back (n);

    for (int res=2; res<=p; ++res) {
        bool ok = true;
        for (size_t i=0; i<fact.size() && ok; ++i)
            ok &= powmod (res, phi / fact[i], p) != 1;
        if (ok)  return res;
    }
    return -1;
}
```

diophantine.h

Description: TODO

4fcad2, 89 lines

```
ll gcd(ll a, ll b, ll& x, ll& y) { //gcd extended
    if (b == 0) {
        x = 1;
        y = 0;
        return a;
    }
    ll x1, y1;
    ll d = gcd(b, a % b, x1, y1);
    x = y1;
    y = x1 - y1 * (a / b);
    return d;
}

bool find_any_solution(ll a, ll b, ll c, ll &x0, ll &y0, ll &g)
    {
    g = gcd(abs(a), abs(b), x0, y0);
    if (c % g) {
        return false;
    }

    x0 *= c / g;
    y0 *= c / g;
    if (a < 0) x0 = -x0;
    if (b < 0) y0 = -y0;
    return true;
}
```

```
void shift_solution(ll & x, ll & y, ll a, ll b, ll cnt) { //
     make sure that a,b coprime
    x += cnt * b;
    y -= cnt * a;
}

ll find_all_solutions(ll a, ll b, ll c, ll minx, ll maxx, ll
     miny, ll maxy) {
    ll x, y, g;
    if (!find_any_solution(a, b, c, x, y, g))
        return 0;
    a /= g;
    b /= g;

    ll sign_a = a > 0 ? +1 : -1;
    ll sign_b = b > 0 ? +1 : -1;

    shift_solution(x, y, a, b, (minx - x) / b);
    if (x < minx)
        shift_solution(x, y, a, b, sign_b);
    if (x > maxx)
        return 0;
    ll lx1 = x;

    shift_solution(x, y, a, b, (maxx - x) / b);
    if (x > maxx)
        shift_solution(x, y, a, b, -sign_b);
    ll rx1 = x;

    shift_solution(x, y, a, b, -(miny - y) / a);
    if (y < miny)
        shift_solution(x, y, a, b, -sign_a);
    if (y > maxy)
        return 0;
    ll lx2 = x;

    shift_solution(x, y, a, b, -(maxy - y) / a);
    if (y > maxy)
        shift_solution(x, y, a, b, sign_a);
    ll rx2 = x;

    if (lx2 > rx2)
        swap(lx2, rx2);

    ll lx = max(lx1, lx2);
    ll rx = min(rx1, rx2);
    if (lx > rx)
        return 0;

    return (rx - lx) / abs(b) + 1;
}
//number of solution of ax + by = c in general
ll number_of_solution(ll a, ll b, ll c, ll x1, ll x2, ll y1, ll
      y2){
    if(a == 0 and b == 0 and c == 0)return (x2-x1+1)*(y2-y1+1);
    else if(a == 0 and b == 0) return 0;
    else if(a == 0){
        if(c%b!=0 or y1>c/b or y2<c/b)return 0;
        else return (x2-x1+1);
    } else if(b == 0){
        if(c%a!=0 or x1>c/a or x2<c/a)return 0;
        else return (y2-y1+1);
    } else {
        return find_all_solutions(a, b, c, x1, x2, y1, y2);
    }
}
```

2.3 Data Structures & Algorithms

cellul4r (3)

Ohm (4)

Techniques (A)

techniques.txt 159 lines

```
Recursion
Divide and conquer
  Finding interesting points in N log N
Algorithm analysis
  Master theorem
  Amortized time complexity
Greedy algorithm
  Scheduling
  Max contiguous subvector sum
  Invariants
  Huffman encoding
Graph theory
  Dynamic graphs (extra book-keeping)
  Breadth first search
  Depth first search
  * Normal trees / DFS trees
  Dijkstra's algorithm
  MST: Prim's algorithm
  Bellman-Ford
  Konig's theorem and vertex cover
  Min-cost max flow
  Lovasz toggle
  Matrix tree theorem
  Maximal matching, general graphs
  Hopcroft-Karp
  Hall's marriage theorem
  Graphical sequences
  Floyd-Warshall
  Euler cycles
  Flow networks
  * Augmenting paths
  * Edmonds-Karp
  Bipartite matching
  Min. path cover
  Topological sorting
  Strongly connected components
  2-SAT
  Cut vertices, cut-edges and biconnected components
  Edge coloring
  * Trees
  Vertex coloring
  * Bipartite graphs (=> trees)
  * 3^n (special case of set cover)
  Diameter and centroid
  K'th shortest path
  Shortest cycle
Dynamic programming
  Knapsack
  Coin change
  Longest common subsequence
  Longest increasing subsequence
  Number of paths in a dag
  Shortest path in a dag
  Dynprog over intervals
  Dynprog over subsets
  Dynprog over probabilities
  Dynprog over trees
  3^n set cover
  Divide and conquer
  Knuth optimization
  Convex hull optimizations
  RMQ (sparse table a.k.a 2^k-jumps)
  Bitonic cycle
  Log partitioning (loop over most restricted)
Combinatorics
  Computation of binomial coefficients
  Pigeon-hole principle
  Inclusion/exclusion
  Catalan number
  Pick's theorem
Number theory
  Integer parts
  Divisibility
  Euclidean algorithm
  Modular arithmetic
  * Modular multiplication
  * Modular inverses
  * Modular exponentiation by squaring
  Chinese remainder theorem
  Fermat's little theorem
  Euler's theorem
  Phi function
  Frobenius number
  Quadratic reciprocity
  Pollard-Rho
  Miller-Rabin
  Hensel lifting
  Vieta root jumping
Game theory
  Combinatorial games
  Game trees
  Mini-max
  Nim
  Games on graphs
  Games on graphs with loops
  Grundy numbers
  Bipartite games without repetition
  General games without repetition
  Alpha-beta pruning
Probability theory
Optimization
  Binary search
  Ternary search
  Unimodality and convex functions
  Binary search on derivative
Numerical methods
  Numeric integration
  Newton's method
  Root-finding with binary/ternary search
  Golden section search
Matrices
  Gaussian elimination
  Exponentiation by squaring
Sorting
  Radix sort
Geometry
  Coordinates and vectors
  * Cross product
  * Scalar product
  Convex hull
  Polygon cut
  Closest pair
  Coordinate-compression
  Quadtrees
  KD-trees
  All segment-segment intersection
Sweeping
  Discretization (convert to events and sweep)
  Angle sweeping
  Line sweeping
  Discrete second derivatives
Strings
  Longest common substring
  Palindrome subsequences
  Knuth-Morris-Pratt
  Tries
  Rolling polynomial hashes
  Suffix array
  Suffix tree
  Aho-Corasick
  Manacher's algorithm
  Letter position lists
Combinatorial search
  Meet in the middle
  Brute-force with pruning
  Best-first (A*)
  Bidirectional search
  Iterative deepening DFS / A*
Data structures
  LCA (2^k-jumps in trees in general)
  Pull/push-technique on trees
  Heavy-light decomposition
  Centroid decomposition
  Lazy propagation
  Self-balancing trees
  Convex hull trick (wcipeg.com/wiki/Convex_hull_trick)
  Monotone queues / monotone stacks / sliding queues
  Sliding queue using 2 stacks
  Persistent segment tree
```